Para criar um syscall novo, são 4 passos. O exemplo abaixo é um syscall `gettime`, que devolve os ticks desde o boot.

1. Número, no `syscall.h` do kernel

Escolha um número livre (o último em uso é `0x004F`) e acrescente a linha:

```
#define SYS_GETTIME             0x0050
```

2. Implementação, no `trap0.c` do kernel

Acrescente um `case` no `switch`, antes do `default`. Os argumentos chegam em `a1`, `a2`, `a3`..., e o valor que você retorna volta em D0 para o app:

```
    case SYS_GETTIME:
        return (int)get_ticks();     /* sua função do kernel */
```

Se a função é de outro arquivo, declare-a com `extern` no topo do `trap0.c`.

3. Wrapper para os apps, no `tools/gen_librigel.py`

Procure a lista TABLE e acrescente uma linha neste formato:

```
("GETTIME", "proc", "gettime", "void"),
```

Os quatro campos são:

- **`GETTIME`**: o nome depois de SYS_ no `syscall.h`.
- **`"proc"`**: em qual header o protótipo cai (`proc.h`, `file.h`, `tty.h`...).
- **`"gettime"`**: o nome da função no app, que vira `rigel_gettime`.
- **`"void"`**: os argumentos em C. Para dois argumentos, por exemplo, use `"long a, char *s"`.

4. Regenerar, recompilar o kernel e os apps

```
cd .../src/librigel
make SYSCALL_H=.../kernel/include/sys/syscall.h
VFS_H=.../kernel/include/sys/vfs.h
```

Depois recompile o kernel e os apps.

No app, você usa assim:

```
#include <rigel/rigel.h>
long t = rigel_gettime();
```

Se algo der errado:

- O gerador mostra `AVISO: SYS_GETTIME esta no syscall.h mas nao na TABELA`. Isso quer dizer que você esqueceu o passo 3.
- O app devolve -1: você esqueceu o `case` do passo 2. O `default` do `trap0.c` devolve -1.

- Se o compilador reclamar de um `proc.h` antigo, é o problema do `-I` que vimos antes.

Fique com o gerador pelo menos enquanto os números estiverem em dois lugares. Ele existe para o número do app nunca divergir do número do kernel, porque ele lê o mesmo `syscall.h`.